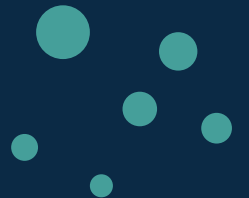


DENSITY-BASED CLUSTERING

DBSCAN

Discovering Clusters of Arbitrary Shape & Detecting Outliers

Prepared by Nancy Ahmed



What is DBSCAN?



Density-Based Spatial Clustering of Applications with Noise

An **unsupervised learning** algorithm used for clustering data.

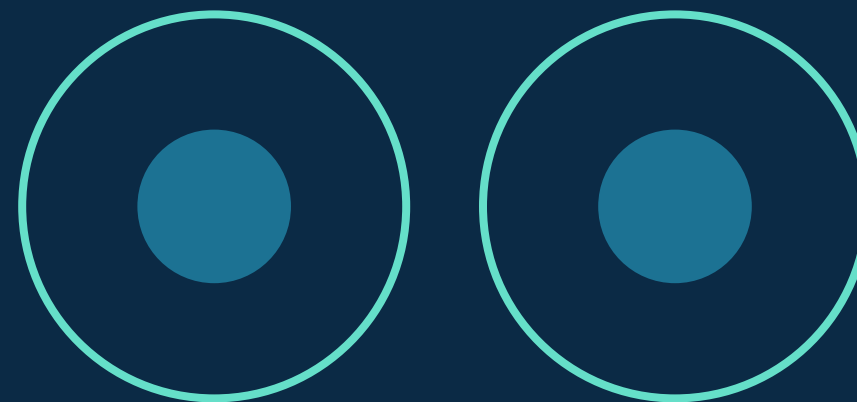
Used for:

- Grouping similar points
- Detecting anomalies / outliers
- Discovering clusters of any shape



Why Not K-Means?

K-Means forces spherical clusters using centroids — and fails on irregular shapes like rings or curves.



Two ring-shaped clusters — DBSCAN identifies both correctly

The Core Idea

**Areas with many nearby points form a CLUSTER.
Sparse areas are considered NOISE.**



Centroid-Based (K-Means)

Clusters defined by distance to a central point



Density-Based (DBSCAN)

Clusters defined by how packed points are together



Outcome

DBSCAN finds clusters of any shape and flags isolated noise points

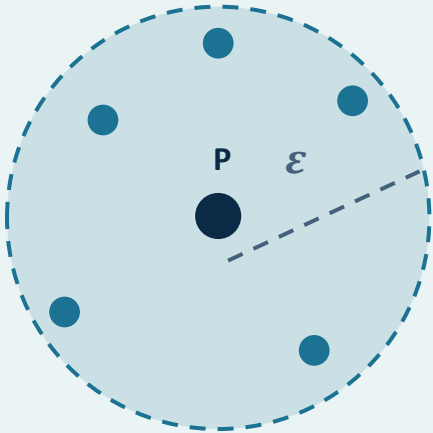
Key Hyperparameters

DBSCAN relies on just two hyperparameters



ϵ (Epsilon)

The radius around a point that defines its “neighborhood.”
Answers: “How far should I look for neighbors?”



MinPts

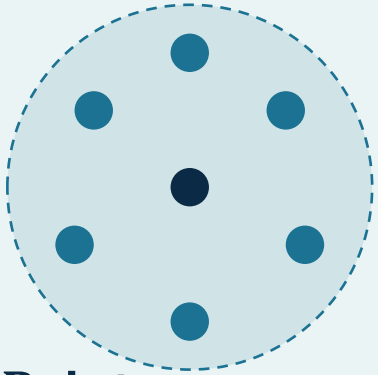
The minimum number of points required inside a point's ϵ -neighborhood for it to be considered “dense.”

Example: MinPts = 5 $\rightarrow$ a point needs at least 5 neighbors nearby to be considered dense.

Together, ϵ and MinPts define what “dense” means for the dataset.

Types of Points

The most important concept in DBSCAN — $\epsilon = 2$, $MinPts = 4$



Core Point

Has at least $MinPts$ neighbors within ϵ .
Forms the dense interior of a cluster.



Border Point

Doesn't have enough neighbors itself,
but lies within ϵ of a Core Point.



Noise Point

Neither core nor border — an isolated
point that belongs to no cluster.

How DBSCAN Works



Density Reachability

Point B is density-reachable from A if A is a core point and B lies within A's ϵ -neighborhood.



Density Connectivity

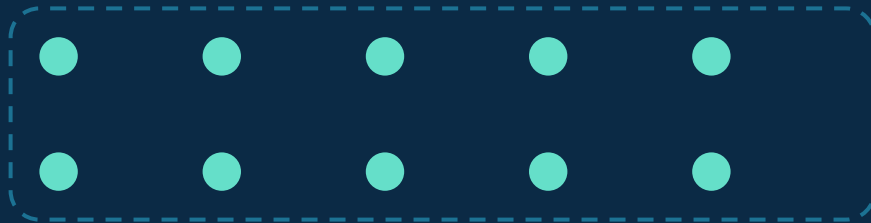
Two points belong to the same cluster if they're connected through a chain of core points.



Example Walkthrough

$\epsilon = 2$ MinPts = 4

Dataset



Dense area → Cluster

Single point → Noise (not enough neighbors)

Result

Top group:

Dense area → forms one cluster

Single point:

Not enough neighbors → classified as noise

Choosing ϵ and MinPts

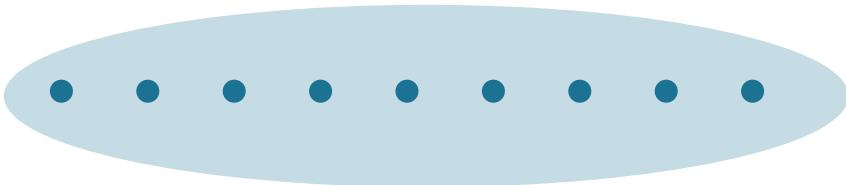
Choosing ϵ

Too Small



Many small clusters & many noise points — under-clustering

Too Large



Everything merges into one cluster — over-clustering



Choosing MinPts

2D data: MinPts = 4

Higher dimensions: MinPts $\geq$ features + 1 (often 5, 10, or 20)



K-Distance Graph

Best method to pick ϵ : compute distance to the k-th nearest neighbor for each point, sort, plot, and look for the 'elbow' — that value is a good ϵ .

Advantages & Disadvantages



Advantages

1

No need to specify K

Automatically discovers the number of clusters, unlike K-Means.

2

Detects outliers

Noise points are naturally identified as part of the algorithm.

3

Handles arbitrary shapes

Finds circular, curved, and irregular clusters.

4

Robust to noise

Performs well even with noisy datasets.



Disadvantages

1

Sensitive to ϵ

A poorly chosen ϵ gives bad results — too small or too large changes everything.

2

Struggles with varying densities

A single ϵ may not work for clusters of different densities.

DBSCAN vs. Other Methods

DBSCAN vs K-Means

Feature	K-Means	DBSCAN
Requires K	Yes	No
Detects Noise	No	Yes
Arbitrary Shapes	No	Yes
Centroid Based	Yes	No
Density Based	No	Yes



DBSCAN's defining strength is that it adapts to the data's density — rather than forcing data into a predefined number of spherical groups.

DBSCAN vs Hierarchical Clustering

DBSCAN	Hierarchical
Density based	Distance based
Finds noise	No noise concept
Faster on large data	Usually slower
No dendrogram	Produces dendrogram

Python Implementation



```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

# Scale features
X_scaled = StandardScaler().fit_transform(X)

# Apply DBSCAN
db = DBSCAN(
    eps=0.5,
    min_samples=5
)

labels = db.fit_predict(X_scaled)
print(labels)
```



Why Scaling Matters

DBSCAN relies entirely on distance — features with larger ranges (e.g. Salary) dominate the distance calculation and distort clusters.

Feature	Value
Age	25
Salary	5000

Standardization: $z = (x - \mu) / \sigma$

Real-World Applications



Fraud Detection

Fraudulent transactions appear as outliers (noise points).



Network Intrusion Detection

Abnormal traffic patterns get flagged as anomalies.



GPS & Geographic Data

Discovering regions of activity from location data.



Customer Segmentation

Finding natural groups of customers by behavior.



Image Processing

Grouping pixels or objects within an image.



Anomaly Detection

Medical and industrial monitoring systems.

Questions & Practice



Quick-Fire Q&A

Q: What does DBSCAN stand for?

A: Density-Based Spatial Clustering of Applications with Noise.

Q: Is DBSCAN supervised?

A: No — it's an unsupervised algorithm.

Q: Does DBSCAN require K?

A: No. It discovers the number of clusters automatically.

Q: What is a core point?

A: A point with at least MinPts neighbors within ϵ .

Q: What is the main weakness?

A: Choosing ϵ and handling clusters with varying densities.

KEY TAKEAWAY

“A cluster is simply a region where points are densely packed together.”



Core points
create clusters



Border points
join them



Isolated points
become noise